1 Introduction

This assignment is divided into three parts. In the first part of this assignment, it is expected that you build a CNN model from scratch and evaluate it in a public dataset. In the second part, it is expected that you train and test two well-known networks and use their output for image classification, using a middle-fusion strategy. The final part consists of an image-retrieval implementation using the best model from the second part, and a metric learning retrain of the embedding space.

This assignment should be done in groups of 2 students max, and it is part of your continuous evaluation. You should provide a report, with no more than 10 pages, and a notebook (.ipynb/.py) which includes your development and analysis for the requested tasks. Structure your report in accordance with the assignment parts.

2 Datasets

In this assignment, you will be using the CIFAR-10[1] dataset and the STL-10[2] dataset. The CIFAR-10 dataset is used for image classification and is composed of a diverse number of objects, animals and other entries, totaling 10 distinct classes (plane, car, bird, cat, deer, dog, frog, horse, ship and truck). For each class there are 6000 colored (RGB) images ($3 \times 32 \times 32$, i.e. 3-channel color images of 32×32 pixels), where 5000 images are used for training and 1000 images are used only for testing. Similarly, the STL-10 dataset is used for image classification and was designed to evaluate unsupervised and semi-supervised learning methods. It consists of 10 distinct classes (airplane, bird, car, cat, deer, dog, horse, monkey, ship and truck), similar to those in CIFAR-10. The dataset contains RGB images of higher resolution, each sized $3 \times 96 \times 96$ (i.e. 3-channel color images of 96×96 pixels). Unlike CIFAR-10, STL-10 provides a limited number of labeled images (a total of 5000 for training and 8000 for testing) along with an additional 100,000 unlabeled images intended for unsupervised feature learning. Considering the smaller dataset size (unlabeled samples are not relevant for this assignment), STL-10 models trained from scratch should present poor performances. Validation splits are not included in the datasets.

2.1 Downloading the datasets

To download the CIFAR-10 dataset, you can use the following code:

```
from torchvision.datasets import CIFAR10
import torchvision.transforms as tt
import numpy as np

transform = tt.ToTensor()

training_data = CIFAR10(download=True,root='./data',transform=transform)
testing_data = CIFAR10(root='./data',train=False,transform=transform)
train_set, val_set = torch.utils.data.random_split(training_data, [int(len(training_data)*0.8), int(
    len(training_data)*0.2)])
```

To download the STL-10 dataset, you can use the following code:

```
from torchvision.datasets import STL10
import torchvision.transforms as tt

transform = tt.ToTensor()

training_data = STL10(root='./data',  split='train', download=True,   transform=transform)
testing_data = STL10(root='./data', split='test', download=True,   transform=transform)
train_set, val_set = torch.utils.data.random_split(training_data, [int(len(training_data)*0.8), int(
    len(training_data)*0.2)])
```

[1] https://www.cs.toronto.edu/~kriz/cifar.html
[2] https://cs.stanford.edu/~acoates/stl10/

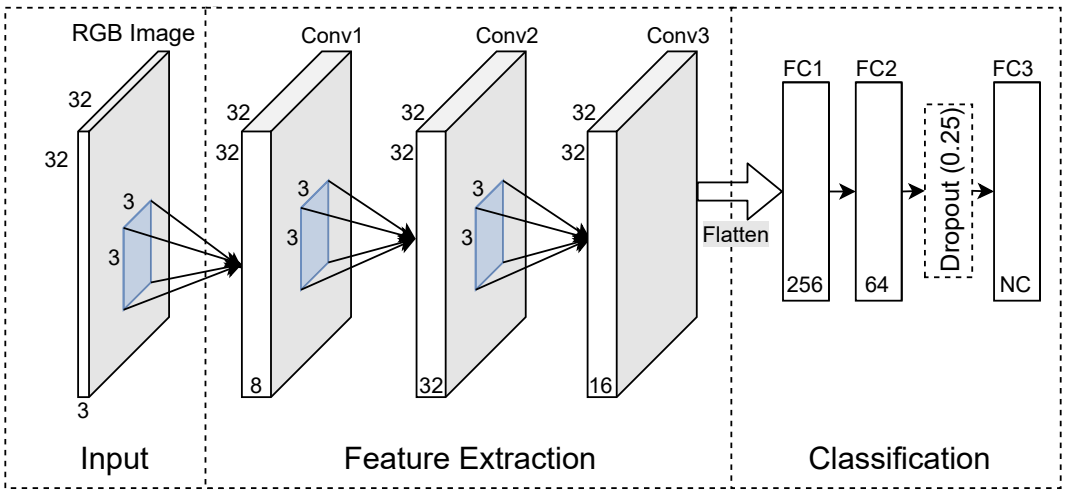


Figure 1: Convolutional Neural Network baseline architecture. NC represents the training number of classes.

3 Network

Figure 1 shows a simple configuration of a CNN architecture. It is composed of three convolutional layers followed by three fully-connected layers. Using the following hyperparameters (lr = 0.0001; epochs = 25), this network achieved 64% of mean-Class Accuracy on the CIFAR-10 dataset.

3.1 Evaluation Metrics

Evaluating a CNN model is a crucial part in the design of an effective learning model. In classification tasks, mean-Class Accuracy (mCA) is the most used evaluation metric and is represented as follows:

$$mCA = \frac{1}{N}\sum_{i=1}^{N}\frac{Correct_i}{Total_i}$$

where $Correct_i$ is the number of correctly predicted samples of class i, $Total_i$ is the total number of samples of class i, and N is the total number of classes.

Other important evaluation metrics include the F1-Score, Precision and Recall, which were already addressed in a previous class. For a better analysis of the trained model, the confusion matrix should also be used.

For the image retrieval task, the quality of the learned feature representations is evaluated by comparing each query image against all images in the gallery. The similarity between feature embeddings is computed using cosine similarity, and the gallery images are ranked in descending order of similarity. The class labels are not used during the retrieval process and are only considered for evaluation purposes. A retrieved image is considered relevant if it belongs to the same class as the query image.

The retrieval performance is evaluated using Precision@K and mean Average Precision (mAP). Precision@K measures the proportion of relevant images among the first K retrieved results and is defined as

$$\text{P@K} = \frac{1}{K}\sum_{i=1}^{K}\text{rel}(i), \tag{1}$$

where $\text{rel}(i) = 1$ if the image retrieved at rank i belongs to the same class as the query image, and $\text{rel}(i) = 0$ otherwise. In this work, Precision@1 and Precision@5 are considered, providing a direct measure of the quality of the highest-ranked retrieval results.

To evaluate the complete ranking, Average Precision (AP) is computed for each query as

$$\mathrm{AP} = \frac{1}{N_{\mathrm{rel}}} \sum_{k=1}^{N} \mathrm{P@k\,rel}(k), \tag{2}$$

where N is the number of images in the gallery and N_{rel} represents the total number of relevant gallery images for the considered query. AP therefore rewards relevant images appearing at higher positions in the ranking.

Finally, mean AP is obtained by averaging the AP over all N_q query images:

$$\mathrm{mAP} = \frac{1}{N_q} \sum_{q=1}^{N_q} \mathrm{AP}_q. \tag{3}$$

While these metrics quantitatively evaluate class-based retrieval, they do not directly measure visual similarity between images belonging to the same class. For this reason, the quantitative evaluation should be complemented with a qualitative analysis of the Top-K retrieved images, considering aspects such as object appearance, pose, viewpoint, and background similarity.

4 Tasks

Part I

- Modify the baseline network (Fig. 1) to reach an output mCA of at least **85%** on the CIFAR-10 dataset. To accomplish this, you can:
  - add/remove convolutional layers, fully-connected layers, batch normalization layers, max-pooling layers, activation functions, and dropout or add other regularization methods;
  - change the loss function and/or change the optimization method (or add a learning rate scheduler);
  - change any hyperparameter;
  - your network cannot have more than **six** convolutional layers;
  - add data augmentation to your dataset.
- Implement the above-mentioned evaluation metrics to evaluate your network.
- Gather the metrics and loss curves for four different hyperparameter combinations (learning rate, batch size, etc)

PART II

- Implement in your code one ResNet model (18, 34, 50, 101 or 152) and one MobileNet V3 (small or large) model (you can import the networks from PyTorch - torchvision.models[3] or timm[4]). Take into account your computational resources in this step, as well as the implications of this choice e.g., choosing one complex architecture in detriment of a simpler architecture, for this specific dataset;
- Define a training/validation split for the STL-10 dataset (e.g., 80/20);
- **Train** both networks you selected from scratch (both networks should be trained using the same training hyperparameters);
- **Train** both networks using, as a starting point, the publicly available pretrained model of the network you are using, in most cases it will be an ImageNet-trained model (check torchvision or timm pretrained weights);

[3] https://docs.pytorch.org/vision/main/models.html
[4] https://huggingface.co/docs/timm/index

- Using the two best trained models (one ResNet and one MobileNetV3) implement a middle fusion technique. You should create a new network that combines the feature maps from the two different models, and create a new classification layer that takes as input the feature maps. The feature maps can be aggregated or concatenated. For each model you should freeze the feature extraction weights (otherwise explain why).

PART III

- Using the best-performing ResNet and MobileNetV3 models obtained in Part II, implement an **image retrieval** system using the STL-10 dataset. Remove the final classification layer of the network and use the features produced as image embeddings;
- Divide the STL-10 test set into two non-overlapping subsets: a **query set** and a **gallery set**. Each image from the query set should be compared against all images in the gallery. Split the test into 20% query and 80% gallery, split independently within each class, and set a fixed seed;
- For each query image, compute the similarity between its embedding and the embeddings of all gallery images using **cosine similarity**. Rank the gallery images according to their similarity and retrieve the Top-K most similar images;
- The class labels must not be used during the retrieval process. They should only be used for training the metric-learning models and for evaluation. For this task, a gallery image is considered relevant if it belongs to the same class as the query image;
- Evaluate the retrieval performance obtained directly from the feature representations learned for the classification task in Part II. Report Precision@1, Precision@5, mean Average Precision (mAP).
- Using the same backbone architectures, train or fine-tune the feature representations using a **metric-learning objective**. Implement at least one of the following approaches:
  - **Triplet Loss**: construct triplets composed of an anchor image, a positive image belonging to the same class, and a negative image belonging to a different class[5];
  - **Contrastive Loss**: construct positive image pairs from the same class and negative pairs from different classes.
- The models obtained in Part II may be used as initialization for the metric-learning stage. Again, the final classification layer should be removed, and the network should be optimized directly using the selected metric-learning objective;
- Repeat the image retrieval evaluation using the embeddings obtained after metric-learning training and compare the results against the original classification embeddings using Precision@1, Precision@5 and mAP;
- Qualitatively evaluate both approaches by presenting a selection of query images together with their Top-5 retrieved images. Indicate which retrieved images belong to the same class as the query and discuss whether the learned representations capture meaningful visual similarities, such as object appearance, pose, viewpoint, or background. In summary, compare the embeddings obtained from the classification models, and the embeddings obtained using Triplet or Contrastive Loss.
- Discuss how the training objective affects the organization of the embedding space and the resulting retrieval performance. In particular, analyze whether the model with the best classification performance also produces the best retrieval representation and whether explicitly optimizing the embedding space improves the retrieval results.

IMPORTANT: Implement the aforementioned evaluation metrics to evaluate your networks, middle fusion strategy, and image retrieval pipeline (e.g., check sklearn); gather the metrics and loss curves for all training conditions.

[5] https://docs.pytorch.org/docs/2.14/generated/torch.nn.modules.loss.TripletMarginLoss.html

Tip: To avoid Colab virtual machine connection issues, save your models after the training phase is completed. To save your CNN models, you can save them on your Google Drive or download them to your machine.

5 Deliverables

Each group (max two students) must deliver the following material **(Deadline 26-10-2026, inforestudante submission)**:

- A detailed report **not exceeding 10 pages** (you could follow an IEEE two-column template for a more compact report) with:
  - **PART I**
    - Diagrams (e.g., draw.io) with the implemented CNN architecture accompanied by descriptions and the thought process when designing the network. You should write any additional implementation details as well as any diagrams you see fit to highlight the different stages of your implementation (e.g., data loading, training, validation, and evaluation). **Hyperparameter values should also be included when relevant (same for parts II and III)**;
    - A table with mCA, F1-Score, Precision, Recall, a plot with the achieved loss curves, and a confusion matrix for the best achieved result;
    - A comparison and a detailed analysis of the results obtained using four different hyperparameters (consider possible overfitting or underfitting scenarios).
  - **PART II**
    - Describe the training protocol for both networks using pretrained weights and when trained from scratch;
    - Evaluation metrics, loss curves, and confusion matrices for the ResNet (pretrained and scratch) and MobileNetV3 (pretrained and scratch) models; Due to the possible computational burden of both methods, and Colab limitations, you can select a suitable number of training epochs (should be the same for all methods).
    - Present the middle fusion approach followed and the obtained results (can be included in the results reported in the previous item, i.e., a single table with all metrics). Include a diagram to represent the middle fusion approach;
    - Detailed comparative analysis of the results obtained.
  - **PART III**
    - Describe the image retrieval pipeline, including the selected feature representation, query/gallery split, similarity metric, and Top-K retrieval procedure;
    - Report the quantitative retrieval results using Precision@1, Precision@5, and mean Average Precision (mAP);
    - Compare the retrieval performance obtained using:
      - the feature embeddings learned for image classification in Part II;
      - the embeddings obtained after training/fine-tuning with Triplet Loss or Contrastive Loss.
    - Present qualitative retrieval examples showing representative query images and their Top-5 retrieved gallery images, clearly identifying correct and incorrect class matches;
    - Include two examples of successful and unsuccessful retrieval cases and discuss the visual characteristics that may have influenced the similarity between images;
    - Compare the representations and discuss whether the model with the highest classification performance also provides the best retrieval performance;
    - Provide a critical analysis of the effect of the metric-learning objective on the learned embedding space and on the final image retrieval performance.
- Google Colab notebook (.ipynb) or python files (.py)